LangGraph ReAct Agent (실습)

5-1. 도구 준비 (실습)
5-2. ReAct Agent 조립 (실습)
5-3. 동작 확인 (실습)
5-4. 추가로 해볼 만한 것
5-5. ReAct Agent와 직접 그래프 중 무엇을 고를 것인가

5-1. 도구 준비 (실습)

> 성격이 다른 도구 3개를 정의해 도구 선택 과정을 관찰할 준비를 해봅니다.

이제 4번째 세션의 수동 루프를 LangGraph로 다시 만듭니다. 코드는 오히려 짧아지고 구조는 더 명확해집니다.

도구 선택 과정을 관찰하려면 성격이 뚜렷하게 다른 도구가 필요합니다.

비슷한 도구 3개면 모델이 헷갈려 하는 모습만 보게 됩니다. 성격이 다른 도구 3개면 모델이 질문을 분해해 알맞은 도구를 고르는 과정이 선명하게 보입니다.

세 가지 성격으로 준비합니다.

1. 조회형: `search_employee`. 임직원 정보를 더미 데이터에서 조회합니다.
2. 계산형: `calculate`. 수식을 받아 정확한 산술 결과를 반환합니다.
3. 판단형: `classify_request`. 도구 내부에서 LLM을 호출해 사내 문의의 담당 부서를 분류합니다. 도구 안에도 AI가 들어갈 수 있다는 것을 보여 주는 도구입니다.

따라하기

1. 새 파일 `day3_07_react_agent.py`를 만들고 도구 3개를 정의합니다.

```
# day3_07_react_agent.py (1/3) - 도구 준비
from typing import TypedDict, Annotated, Literal
from dotenv import load_dotenv
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode
from langchain_aws import ChatBedrockConverse
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage, SystemMessage

load_dotenv()
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)


def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니
다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinsta
nce(block, dict))
    return content


@tool
def search_employee(name: str) -> str:
    """임직원 이름으로 소속 팀과 이메일을 조회합니다. 임직원 정보 질문에 사용하세요."""
    # 교육용 더미 데이터입니다. 실제 사내 데이터와 무관합니다.
    employees = {
        "김하늘": {"team": "클라우드운영팀", "email": "haneul.kim@samsungsds.examp
le.com"},
        "박도윤": {"team": "물류플랫폼팀", "email": "doyun.park@samsungsds.exampl
e.com"},
    }
    if name in employees:
        e = employees[name]
        return f"{name} / {e['team']} / {e['email']}"
    return f"'{name}' 님을 찾을 수 없습니다. 조회 가능한 임직원: {list(employees.keys
())}"


@tool
def calculate(expression: str) -> str:
    """수식 문자열을 계산해 정확한 결과를 반환합니다. 예: '3480000 * 1.1', '48 * 1.1
```

```
2"""
    try:
        return str(eval(expression, {"__builtins__": {}}, {}))
    except Exception as e:
        return f"계산 실패: {e}. 수식 형식을 확인하세요."


@tool
def classify_request(text: str) -> str:
    """사내 문의 문장을 받아 담당 부서를 IT지원, 인사, 총무 중 하나로 분류합니다. 문의 접수와 부서 배정 질문에 사용하세요."""
    # 도구 안에서 다시 LLM을 호출합니다. 도구 안의 AI입니다.
    result = llm.invoke(
        "다음 사내 문의의 담당 부서를 IT지원, 인사, 총무 중 하나로만 답하세요. "
        f"다른 말은 붙이지 마세요.\n\n문의: {text}"
    )
    return get_text(result).strip()


tools = [search_employee, calculate, classify_request]
```

2. 도구를 그래프 없이 단독으로 실행해 동작을 확인합니다. 파일 끝에 임시로 추가했다가 확인 후 지웁니다.

```
print(search_employee.invoke({"name": "김하늘"}))
print(calculate.invoke({"expression": "48 * 1.12"}))
print(classify_request.invoke({"text": "노트북 화면이 계속 깜빡거려요"}))
```

막힐 때

- `@tool`을 붙인 함수를 `search_employee("김하늘")` 처럼 직접 호출하면 에러가 납니다.

  데코레이터가 함수를 Tool 객체로 바꿨기 때문입니다. 단독 실행은 `search_employee.invoke({"name": "김하늘"})` 형태로 합니다.

- `classify_request`가 부서 이름 외의 말을 붙여 반환합니다.

  프롬프트의 "다른 말은 붙이지 마세요"가 핵심입니다. 그래도 불안정하면 temperature=0인 모델 객체를 별도로 만들어 씁니다.

- docstring을 어디까지 써야 할지 감이 안 옵니다.

  어떤 상황에 이 도구를 쓰라는 것까지 적습니다. 인자가 없는 도구라도 docstring은 반드시 갖춰야 모델이 언제 쓸지 판단할 수 있습니다.

확인: 세 도구의 단독 실행 결과가 각각 임직원 정보, 53.76, IT지원으로 나오면 성공입니다.

조회형, 계산형, 판단형으로 성격이 다른 도구 3개를 준비하고 도구 안에 LLM을 넣을 수 있다는 것도 확인했습니다.

5-2. ReAct Agent 조립 (실습)

4번째 세션의 수동 루프를 LangGraph의 노드 2개짜리 그래프로 재구성해봅니다.

4번째 세션 4-3의 대응표를 그대로 코드로 옮깁니다. 완성될 그래프의 구조는 다음과 같습니다.

따라하기

1. 5-1 파일에 이어서 그래프를 조립합니다.

```
# day3_07_react_agent.py (2/3) - 그래프 조립

# [수동 루프의 messages 리스트] -> State + add_messages reducer
class AgentState(TypedDict):
    messages: Annotated[list, add_messages]


SYSTEM_PROMPT = """너는 삼성SDS 사내 AI 어시스턴트야.

[도구 사용 전략]
1. 임직원 정보는 반드시 search_employee로 조회해. 지어내지 마.
2. 계산이 필요하면 calculate를 사용해. 암산하지 마.
3. 사내 문의의 담당 부서 판단은 classify_request를 사용해.
4. 도구가 필요 없는 인사나 일반 질문에는 도구 없이 바로 답해."""

llm_with_tools = llm.bind_tools(tools)


# [수동 루프의 invoke와 append] -> agent 노드
def agent(state: AgentState) -> dict:
    messages = [SystemMessage(content=SYSTEM_PROMPT)] + state["messages"]
    return {"messages": [llm_with_tools.invoke(messages)]}


# [수동 루프의 if not ai_message.tool_calls] -> 라우팅 함수
def should_continue(state: AgentState) -> Literal["tools", "end"]:
    last = state["messages"][-1]
    return "tools" if last.tool_calls else "end"


builder = StateGraph(AgentState)
builder.add_node("agent", agent)
builder.add_node("tools", ToolNode(tools))   # [도구 실행과 ToolMessage 포장] ->
내장 ToolNode

builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", should_continue, {"tools": "tools",
"end": END})
builder.add_edge("tools", "agent")           # [while 루프] -> Cycle

graph = builder.compile()

# 구조 시각화 (PNG 저장이 안 되는 환경 대비 try/except)
try:
    graph.get_graph().draw_mermaid_png(output_file_path="react_agent.png")
    print("react_agent.png 저장 완료")
except Exception as e:
```

```
    print("PNG 저장은 건너뜁니다:", e)
    print(graph.get_graph().draw_mermaid())
```

2. 코드에서 네 가지를 관찰합니다.
   - 노드 2개가 전부입니다. agent(추론)와 tools(행동과 관찰)로 수동 루프 약 30줄이 이 구조로 정리됐습니다.
   - `ToolNode(tools)`는 LangGraph 내장 노드입니다. 마지막 AIMessage에서 tool_calls를 찾아 도구를 실행하고 그 결과를 ToolMessage로 만들어 messages에 추가하는 일을 전부 대신합니다. 4번째 세션 4-2에서 손으로 짠 for 블록 전체가 이 한 줄입니다.
   - agent 노드가 SystemMessage를 State에 넣지 않고 호출 직전에 앞에 붙입니다. 시스템 프롬프트는 매 호출의 지시일 뿐 대화 기록의 일부가 아니므로 State에 남기지 않습니다.
   - `builder.add_edge("tools", "agent")` 한 줄이 Cycle입니다. 도구 실행이 끝나면 무조건 agent로 돌아가 재판단합니다.

막힐 때

- `ToolNode` import 에러가 납니다.

  경로는 `from langgraph.prebuilt import ToolNode`입니다. `langgraph.graph`가 아닙니다.

- 라우팅 함수가 "end"를 반환하는데 실행 중 에러가 납니다.

  매핑 dict에 "end" 키가 없는 경우입니다. 라우팅 함수의 반환값 집합과 매핑 dict의 키 집합은 정확히 일치해야 합니다.

- State에 messages 외의 필드를 추가하고 싶어집니다.

  ReAct에서는 대부분 필요 없습니다. 모든 문맥이 AIMessage와 ToolMessage로 messages 안에 쌓이기 때문입니다. 재시도 횟수 같은 별도 카운터가 정말 필요할 때만 추가합니다.

확인: 파일을 실행했을 때 에러 없이 그래프가 컴파일되고 PNG 파일 또는 mermaid 텍스트로 agent와 tools 사이의 순환 구조가 보이면 성공입니다.

> State, 조건부 엣지, ToolNode, Cycle 네 부품으로 노드 2개짜리 ReAct Agent를 완성했습니다.

5-3. 동작 확인 (실습)

> 도구 미사용, 단일 도구, 복합 질문으로 도구 선택과 체이닝 과정을 관찰해봅니다.

도구 선택 과정을 관찰하려면 최종 답만 보지 말고 messages에 쌓인 모든 메시지를 출력해야 합니다. 어떤 도구가 어떤 인자로 불렸는지 눈에 보여야 Agent가 정말 스스로 골랐다는 것을 확인할 수 있습니다.

따라하기

1. 5-2 파일에 이어서 관찰용 함수와 질문 세트를 추가하고 실행합니다.

```
# day3_07_react_agent.py (3/3) - 동작 확인
def run_and_trace(question: str):
    print(f"\n{'=' * 60}\n질문: {question}\n{'=' * 60}")
    result = graph.invoke({"messages": [HumanMessage(content=question)]})
    for m in result["messages"]:
        label = type(m).__name__
        if hasattr(m, "tool_calls") and m.tool_calls:
            for tc in m.tool_calls:
                print(f"  [{label}] 도구 요청: {tc['name']}({tc['args']})")
        elif m.content:
            print(f"  [{label}] {get_text(m)[:150]}")


# 경로 1: 도구 미사용 질문
run_and_trace("어떤 일을 도와줄 수 있는지 한 문장으로 소개해줘.")

# 경로 2: 단일 도구 질문 (도구별 1개씩)
run_and_trace("김하늘 님은 어느 팀 소속이야?")
run_and_trace("3480000 곱하기 1.1은 얼마야?")
run_and_trace("'노트북 화면이 계속 깜빡거려요'라는 문의는 어느 부서 담당이야?")

# 경로 3: 복합 질문 (도구 2개 이상 체이닝)
run_and_trace(
    "'프린터가 고장 났어요' 문의를 담당 부서로 분류하고, "
    "그 부서의 이번 달 처리 건수 48건이 12% 늘어나면 몇 건이 되는지도 계산해줘."
)
run_and_trace("박도윤 님 이메일을 알려주고, '연차 이월 기준이 궁금해요' 문의의 담당 부서도 분류
해줘.")
```

2. 복합 질문의 실행 결과 예시입니다.

```
============================================================
질문: '프린터가 고장 났어요' 문의를 담당 부서로 분류하고, 그 부서의 이번 달 처리 건수 48건이 12%
늘어나면 몇 건이 되는지도 계산해줘.
============================================================
  [HumanMessage] '프린터가 고장 났어요' 문의를 담당 부서로 분류하고, ...
  [AIMessage] 도구 요청: classify_request({'text': '프린터가 고장 났어요'})
  [ToolMessage] IT지원
  [AIMessage] 도구 요청: calculate({'expression': '48 * 1.12'})
  [ToolMessage] 53.76
  [AIMessage] '프린터가 고장 났어요' 문의는 IT지원 담당입니다. 처리 건수 48건이 12% 늘어나면
약 54건이 됩니다.
```

3. 세 경로가 각각 다른 것을 보여 줍니다. 경로 1에서는 agent가 첫 턴에 tool_calls 없이 바로 답했고 라우팅 함수가 곧장 END로 보냈습니다. 도구를 안 쓰는 판단도 Agent 품질의 일부입니다.
   모든 질문에 도구부터 들이대는 Agent는 좋은 Agent가 아닙니다.

   경로 2는 agent, tools, agent, END로 딱 한 바퀴 돈 가장 전형적인 ReAct 사이클입니다. 경로 3에서는 분류 결과가 나온 뒤에야 계산을 할 수 있으므로 모델이 도구를 순서대로 체이닝했습니다.
   이 순서를 우리는 코드 어디에도 적지 않았습니다. LLM이 질문을 분해하고 순서를 정한 것입니다. 이것이 ReAct의 핵심 가치입니다.

4. 추가 실험으로 시스템 프롬프트에서 "계산이 필요하면 calculate를 사용해. 암산하지 마." 줄을 지우고 복합 질문을 다시 돌리면 모델이 calculate 없이 암산해 버리는 경우가 생깁니다. 시스템 프롬프트가 도구 사용 행동을 어떻게 바꾸는지 체감할 수 있습니다.

막힐 때

- 같은 질문인데 실행할 때마다 경로가 조금씩 다릅니다.

  정상입니다. 흐름 제어를 LLM이 하므로 같은 질문이라도 경로가 달라집니다(비결정적). 이것이 ReAct의 유연함이자 단점(예측과 디버깅이 어려움)입니다. 흐름을 반드시 고정해야 하는 구간은 3번째 세션의 방식대로 그래프 엣지로 강제합니다. 실무에서는 두 방식을 섞어 씁니다.

- `GraphRecursionError`가 납니다.

  모델이 같은 도구를 계속 반복 호출하고 있을 가능성이 큽니다. 시스템 프롬프트에 재시도 한도를 명시하고 도구가 실패했을 때 다른 인자로 시도하라는 힌트를 반환하게 개선합니다.

- 도구 호출 이력이 안 보입니다.

  `result["messages"]` 전체를 출력하고 있는지 확인합니다. 마지막 메시지만 보면 중간 과정이 안 보입니다. 실시간으로 보고 싶으면 `graph.stream(..., stream_mode="values")`를 씁니다.

확인: 도구 미사용 1건, 단일 도구 3건, 복합 2건에서 의도한 도구가 호출되고 복합 질문에서 도구가 순서대로 체이닝되는 것이 trace에 보이면 성공입니다.

> 같은 Agent가 질문에 따라 도구를 안 쓰기도 하고 하나만 쓰기도 하고 여러 개를 엮기도 한다는 걸 관찰했습니다.

5-4. 추가로 해볼 만한 것

▼ 2일차 RAG Retriever를 도구로 추가

2일차에서 디스크에 저장해 둔 벡터 DB를 그대로 도구로 감싸면 Agent가 사내 정책 문서까지 검색할 수 있게 됩니다. 2일차의 `chroma_db` 폴더를 `day3_practice` 폴더에 복사한 뒤 아래 도구를 추가합니다. langchain-chroma는 1일차 requirements.txt에 이미 들어 있습니다.

```
# 5-1 도구 정의 아래에 추가
from langchain_aws import BedrockEmbeddings
from langchain_chroma import Chroma

embeddings = BedrockEmbeddings(
    model_id="amazon.titan-embed-text-v2:0",
    region_name="us-east-1",
)
vectorstore = Chroma(
    persist_directory="./chroma_db",
    collection_name="sds_policies",
    embedding_function=embeddings,
)


@tool
def search_policy(query: str) -> str:
    """사내 정책 문서를 의미 기반으로 검색합니다. 연차, 재택, 출장 규정 질문에 사용하세요."""
    docs = vectorstore.similarity_search(query, k=3)
    if not docs:
        return "검색 결과가 없습니다."
    return "\n\n---\n\n".join(d.page_content for d in docs)
```

바꿀 곳은 세 군데뿐입니다. `tools` 리스트에 `search_policy`를 추가하고(bind_tools는 tools 리스트를 쓰므로 자동 반영) 시스템 프롬프트의 도구 안내에 한 줄을 더합니다. 그래프 구조는 한 글자도 안 바뀝니다. 새 기능 추가가 곧 도구 추가라는 ReAct의 확장 방식을 확인할 수 있습니다.

"재택근무 규정을 찾아서 알려주고, 김하늘 님 이메일로 안내한다면 누구에게 보내는 건지도 알려줘" 같은 질문으로 RAG 검색과 임직원 조회가 한 번에 엮이는지 확인해 봅니다.

5-5. ReAct Agent와 직접 그래프 중 무엇을 고를 것인가

> ReAct 프리빌트와 직접 그래프 중 무엇을 고를지 판단 기준을 이해해봅니다.

오늘 같은 일을 두 번 했습니다. 4번째 세션에서는 ReAct 루프를 노드와 엣지로 직접 조립했고 5번째 세션에서는 프리빌트 함수 한 줄로 끝냈습니다.

다음에 무엇을 할지를 모델이 정해야 하면 ReAct, 우리가 정해야 하면 그래프입니다.

상황	권장	이유
도구를 여러 개 주고 알아서 골라 쓰게 하면 되는 경우	ReAct 프리빌트	만드는 데 한 줄, 도구 추가도 리스트에 등록만 하면 끝입니다
단계 순서가 업무 규칙으로 정해져 있는 경우 (검증 → 승인 → 실행)	직접 그래프	순서를 모델 판단에 맡기면 건너뛰거나 순서가 바뀝니다
분기 조건을 코드로 명시해야 하는 경우 (금액 100만원 이상이면 승인)	직접 그래프	조건문은 프롬프트가 아니라 코드여야 감사가 됩니다
특정 단계만 다른 모델이나 파라미터를 써야 하는 경우	직접 그래프	노드마다 따로 지정할 수 있습니다
빠른 프로토타입과 시연	ReAct 프리빌트	구조를 고민하는 시간을 도구 품질에 씁니다

실무에선 둘 중 하나를 고르기보다 **섞어 쓰는 경우가 더 많습니다.** 바깥 뼈대는 그래프로 짜서 순서와 승인 지점을 고정하고, 그 안의 한 노드를 ReAct Agent로 두는 식입니다. 6일차의 Supervisor 구조가 정확히 이 모양입니다.

프리빌트를 쓰더라도 4번째 세션에서 손으로 짜 본 경험은 남습니다. 프리빌트가 안 맞는 순간 열어서 고칠 수 있기 때문입니다.

오늘 완성한 ReAct Agent 그래프는 여기서 끝나지 않습니다. 6일차에서 Supervisor가 여러 전문 Agent에게 일을 배분할 때 오늘 만든 그래프가 그 하위 Agent로 그대로 들어갑니다.

> 순서와 승인 지점을 고정해야 하면 그래프, 도구 품질에 집중하려면 프리빌트라는 선택 기준을 익혔습니다.